OBJECT DETECTION USING DETR: IMPROVISING THE BASELINE MODEL

A20549528

SUDHANVI VAKUL VAKULABHARANAM

ECE 508

FINAL PROJECT REPORT

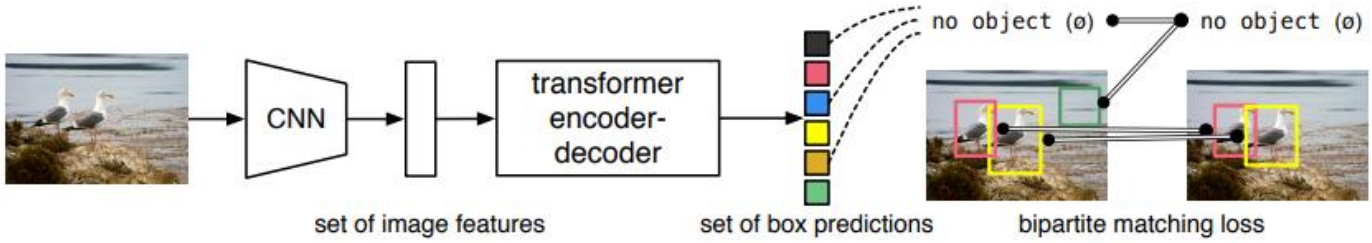
OBJECT DETECTION USING TRANSFORMER(DETR)

Introduction to DETR

A revolutionary change in object detection techniques is represented by Facebook AI Research's DEtection TRansformer (DETR). Conventional detectors such as SSD, YOLO, and Faster R-CNN depend on complex pipelines that integrate convolutional neural networks (CNNs) with other elements including region proposal networks, anchor boxes, and post-processing methods like Non-Maximum Suppression (NMS). Although these architectures have produced impressive results, their intricacy frequently calls for considerable hyperparameter adjustment, which presents scalability and optimization issues.

By presenting object detection as a set prediction problem, DETR redefines the field. Instead of separating individual items for processing separately, it uses a transformer-based architecture to process the entire image as a cohesive group of objects. The reliance on manually created elements, like anchor boxes and region suggestions, that characterize conventional approaches is removed by this end-to-end approach.

The capacity of DETR to anticipate a predetermined number of bounding boxes and class labels at once, which simplifies the detection pipeline, is one of its best qualities. DETR stresses design simplicity and mitigates the risk of repeated predictions by eliminating the requirement for NMS. This change in architecture allows for better efficiency in object detection tasks, particularly for complex scenarios, and lessens the need for manually created parameters.



Motivation Behind DETR

The main goal of DETR is to transform object detection by utilizing transformers' special qualities, especially their capacity to use self-attention processes to describe long-range relationships. Transformers are excellent at collecting global context and relationships within data; they were first created for natural language processing tasks. By applying this power to object detection, DETR overcomes the drawbacks of traditional convolutional neural network (CNN)-based techniques, which frequently find it difficult to understand intricate inter-object relationships dispersed throughout an image.

The restricted receptive fields of conventional CNN-based object detectors, which concentrate on certain areas of an image, constitute a limitation. This concentrated concentration is useful for acquiring minute details, but it makes it difficult to obtain thorough contextual information. CNNs may not correctly perceive circumstances where objects are partially obscured, overlapped, or positioned ambiguously, making this shortcoming especially problematic.

With its transformer design, DETR gets around this restriction by allowing all of the image's elements to interact with one another through self-attention processes. DETR can recognize complex links between items and their surroundings thanks to this global interaction, which makes it easier to comprehend the image holistically. Modeling these global relationships greatly improves detection accuracy, especially in scenarios with a lot of moving objects or unclear visual clues.

Set Prediction Problem in Object Detection

DETR's novel approach to object detection as a set prediction problem is one of its distinguishing characteristics. Depending on the content of the image, traditional object detectors produce varying quantities of predictions. To manage these fluctuating outputs, sophisticated methods like anchor boxes and Non-Maximum Suppression (NMS) are frequently needed. DETR, on the other hand, streamlines this procedure by generating a predetermined quantity of predictions for each image. By doing away with anchor boxes and NMS, this fixed-set method simplifies the detection pipeline and lessens the requirement for manually adjusted parameters.

DETR uses the Hungarian method for bipartite matching between predicted and ground-truth bounding boxes to make set prediction easier. Every predicted box is assigned to a ground-truth

item in a unique way thanks to this effective matching procedure. The model prevents producing duplicate detections during training by reducing overlap and redundancy.

In addition to improving training effectiveness, the Hungarian approach strengthens DETR's end-to-end architecture. Instead of depending on post-processing heuristics, this method enables the model to concentrate on learning meaningful object representations. DETR greatly streamlines the detection process while preserving high accuracy and robustness, even in intricate visual environments, by presenting object detection as a set prediction job.

Core Innovations in DETR

- **Direct Set Prediction Using Transformers**: DETR's handling of object detection as a set prediction problem is a significant departure from traditional methods. Instead of generating a variable number of predictions, DETR generates a fixed set of bounding boxes and class labels for each image. The transformer's self-attention mechanism, which takes into account the relationships between each object in a picture, may allow DETR to replicate dependencies that would be difficult for CNN-based detectors to learn.

- **Anchor-Free and NMS-Free Detection**: DETR does not require anchor boxes, which are commonly used in object detectors to offer potential positions for items. Anchor boxes must be carefully adjusted because they are often a source of complexity in object detection pipelines. Furthermore, DETR does not rely on Non-Maximum Suppression (NMS) to remove duplicate predictions because it generates a fixed set of predictions with different assignments to ground-truth objects using bipartite matching.

- **Global Context Modeling with Transformers**: DETR uses a transformer design to integrate global context modeling into object detection. Because transformers have a self-attention mechanism, any component of the image may "attend" to every other component, allowing DETR to recognize global patterns and dependencies. This ability is very helpful in situations with complex object interactions when understanding the entire context may improve detection accuracy.

- **End-to-End Training**: The network can be optimized collectively since DETR is designed to be trained end-to-end. This architecture contrasts with traditional detectors, which may require different training phases for different components. DETR's end-to-end architecture

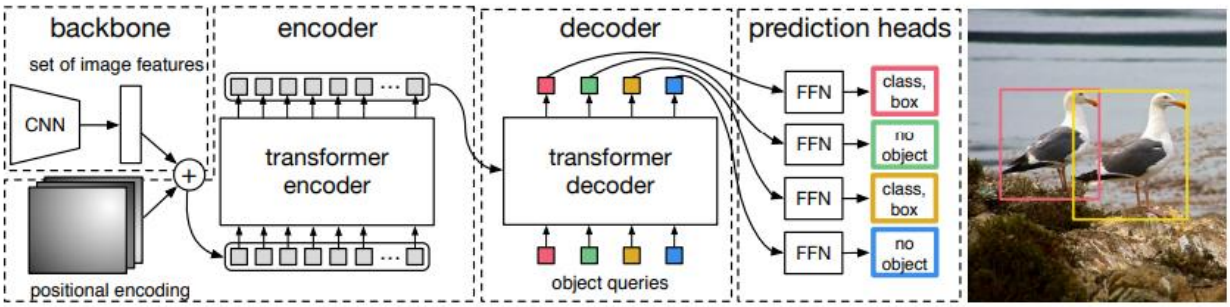
simplifies the training process by removing the requirement for component-specific training or adjustment.

Comparison with Traditional Object Detectors

- **Anchor Boxes**: Traditional models use anchor boxes, which are predetermined bounding boxes with different aspect ratios and scales. These anchor boxes are used as rough estimates for object placement. DETR, on the other hand, does not require anchor boxes and predicts bounding box coordinates directly.
- **Region Proposal Network (RPN)**: In Faster R-CNN and related models, an RPN creates regions of interest, which are further refined by a second stage. Instead of employing a two-step process, DETR uses a transformer encoder-decoder design to anticipate bounding boxes in a single stage.
- **Non-Maximum Suppression (NMS)**: In traditional models, NMS is used to remove repeated detections by retaining only the highest confidence forecast for each object. DETR circumvents the need for NMS by generating a preset set of unique predictions for each image.
- **Self-Attention Mechanism**: Conventional models are based on CNNs, which have limited receptive fields. Because of DETR's transformer design, which captures long-range dependencies and aids the model in comprehending complex relationships between objects, every pixel in the image can attend to every other pixel.

DETR represents a significant shift in object detection methodology by utilizing transformers to provide direct set prediction without the need for anchor boxes or NMS. Because of its global attention method, which allows it to learn context throughout the entire image, it is suitable for scenarios involving complex object interactions. By redefining object detection as a set prediction problem and using bipartite matching for unique prediction assignment, DETR simplifies the detection process and reduces dependence on complex components. These developments have made it possible for DETR to detect objects in an end-to-end manner, providing a more effective, trainable, and adaptable approach for a variety of object recognition applications.

Overall Architecture of DETR



The purpose of the DEtection TRansformer (DETR) architecture is to solve the problem of object detection as a set prediction by utilizing transformer power. The backbone, transformer encoder, transformer decoder, and prediction heads are the architecture's four primary parts. Every component has a distinct function, such as deciphering the input image, obtaining pertinent data, simulating regional dependencies, and predicting item locations and classes.

a. Backbone (Feature Extractor): DETR is based on a convolutional neural network, often a ResNet-50 or ResNet-101 model. The backbone's job is to process the incoming image and extract a feature map that, at a reduced resolution, captures the image's spatial and semantic information. Due to their ability to recognize local patterns and trends, CNNs are a strong choice for producing feature maps from input photographs.

The backbone applies convolutional and pooling layers to the input image in several processing rounds to progressively reduce the spatial resolution and deepen the feature map. Each layer of the CNN records several aspects of the image, such as edges, textures, and complex shapes, which are useful for identifying objects. With CCC representing the number of channels and HHH and WWW representing the height and breadth, respectively, the final feature map of the backbone has a depth of CCC and a spatial resolution of H×WH times WH×W.

The output feature map is then sent from the backbone to the transformer encoder. Before being input into the encoder, the feature map is flattened into a sequence of H×WH times WH×W vectors, each with a length of CCC. This flattening step prepares the feature map for processing because the transformer works with sequences rather than spatial maps.

b. Positional Encoding: Transformers are designed to process sequences and are invariant to the order of the input items. This architecture works well for applications such as natural language processing, where word order does not always have a fixed spatial meaning. However, for computer vision tasks, the spatial relationships between different visual components are essential. To ensure that the transformer maintains spatial information, DETR employs positional encoding.

Every element of the flattened feature map receives extra spatial information through a process known as positional encoding. Positional encodings in DETR are computed using sine and cosine functions of different frequency. By adding these encodings to the flattened feature map, the transformer can then distinguish between different spatial regions in the image. Positional encoding helps the model understand spatial relationships and makes it easier for the transformer to spot patterns relevant to object recognition.

c. Transformer Encoder: The transformer encoder is a sequence-processing module that applies feed-forward layers and self-attention to the feature map. Each of the encoder's several layers consists of two main sublayers: a multi-head self-attention layer and a feed-forward neural network.

Multi-Head Self-Attention: The self-attention mechanism allows each piece in the sequence to interact with every other element. The model is able to comprehend relationships between different visual aspects and record global context thanks to this interaction. In DETR, multi-head self-attention splits the feature map into multiple "heads," each of which picks up a unique type of association. These heads are then concatenated to produce a comprehensive representation of the dependencies among picture regions.

Feed-Forward Neural Network: After the self-attention operation, the output is processed by a feed-forward neural network. This network further processes the representation learned by self-attention, adding non-linearity to improve the model's ability to identify complex patterns. Each layer of the transformer encoder consists of residual connections and normalization layers, which help to stabilize training and improve model performance.

The transformer encoder creates an improved feature representation where each element in the sequence uses information from every other region of the image. This global context helps the

model recognize things more accurately since it takes into account the relationships between distant sites.

d. Decoder for Transformers: The transformer decoder is the next stage of DETR's architecture. The decoder is also composed of multiple layers, each of which has sublayers for feed-forward, cross-attention, and self-attention. The decoder processes a set of learnable embeddings called object queries to produce predictions for potential objects in the image.

Object Queries: Object queries are learnable embeddings that represent potential items in the image. Each object query corresponds to a predicted object, and the number of queries remains constant regardless of the actual number of things in the image. As these queries learn to represent numerous objects and classes during training, they become flexible to different object configurations. Throughout the decoding process, the transformer decoder uses these object queries to manage the encoder's output and forecast bounding boxes and class labels.

Cross-Attention: Cross-attention in the transformer decoder, which gathers information from the entire image to produce predictions, allows each object query to attend to the encoder's output. This cross-attention phase enables each query to identify and classify things based on global context. The object queries interact with the encoder's output and take into consideration the relationships the encoder has found in order to generate predictions.

The transformer decoder generates a preset set of predictions for each object query. Each prediction has a bounding box and a class label. The prediction heads receive the output from the decoder and convert the intermediate representations into the bounding box coordinates and final class probabilities.

e. Prediction Heads: In DETR, the final bounding box and class label predictions are produced by two Multi-Layer Perceptrons acting as the prediction heads. Each prediction head processes the output from the transformer decoder and generates a unique set of predictions for each object query.

Bounding Box Prediction Head: The bounding box prediction head outputs the coordinates of each bounding box. These coordinates show the location and dimensions of the objects that were found. Because DETR predicts bounding boxes directly, without the need of anchor boxes, the bounding box prediction head generates continuous values for the box coordinates. The loss

function of the training procedure helps to align the projected bounding boxes with the ground-truth boxes.

Class Prediction Head: The class prediction head assigns a class label to each projected bounding box. In addition to a "no object" class that accounts for the possibility that a certain query does not correlate to any objects in the image, a probability distribution across all classes is generated. With the help of the class prediction head, the model can distinguish between many object classes and identify areas that are object-free.

Together, the bounding box and class prediction heads produce a predetermined number of predictions for each image. Every forecast has a class label in addition to a bounding box. With this fixed-size output, DETR can avoid the need for NMS since each prediction is uniquely matched to a ground-truth object using bipartite matching.

This architecture simplifies the detection process and enables DETR to perform object detection without the use of anchor boxes, region proposals, or NMS by utilizing transformers to exploit global context.

Workflow of DETR

The Detection Transformer (DETR) goes through several structured phases to identify objects in pictures. The initial stage, referred to as input preprocessing, involves scaling images to a preset resolution and normalizing pixel values. During training, data augmentation techniques including random cropping, flipping, and color jittering are employed to enhance the model's generalization across variations in object scale, orientation, and lighting conditions.

The backbone network, usually a ResNet-50 or ResNet-101, handles the pictures after preprocessing. The backbone uses the spatial and semantic aspects of the image to generate a lower-resolution feature map that includes the essential information for object detection. A sequence of vectors, each representing a different spatial area in the image, is then created by flattening this feature map.

To maintain spatial information, DETR applies positional encodings to the flattened feature map. These encodings, which are derived using sine and cosine functions, allow the transformer to understand the spatial relationships between different regions of the image. The positional

encodings are added to the feature vectors, providing crucial spatial context for the subsequent transformer operations.

The transformer encoder handles the positional encoded feature vectors by employing self-attention techniques. By allowing each piece in the sequence to focus on every other element, this captures global context throughout the image. The encoder creates a better representation of the image, with each vector incorporating information from all spatial regions, to make object detection easier in complex situations.

To the transformer decoder, DETR adds a predetermined collection of learnable embeddings called object queries. These searches act as stand-ins for potential object instances and focus on certain regions of the image. Regardless of the actual number of objects in a picture, the number of object queries remains constant, which simplifies the management of variable object counts during training and inference.

The decoder uses cross-attention to let object queries handle the encoder outputs. By extracting relevant MLP, each query may effectively identify and classify things according on global context. The class prediction head assigns a class label to each bounding box and returns a probability distribution across all possible classes, including a "no object" class for unnecessary searches. This direct set prediction method enables DETR to efficiently and accurately locate objects in images without the need for traditional anchor boxes or Non-Maximum Suppression.

Loss Function and Training Process

In order to prevent duplicate detections, DETR's loss function matches each predicted object to a distinct ground-truth object during model training.

1. Classification Loss: Using cross-entropy loss, the classification loss calculates the difference between the ground-truth labels and the anticipated class probabilities. When addressing queries that shouldn't correspond to any objects, it pushes the model to give proper classes high probabilities.

2. Bounding Box Loss: Bounding box loss uses two components to assess the difference between ground-truth and anticipated boxes:

L1 Loss: To match predicted boxes with object boundaries, it penalizes variations in x, y, width, and height coordinates.

Generalized IoU (GIoU) Loss: Enhances spatial alignment by optimizing bounding box placement by taking overlap and location into account.

3. Hungarian Matching for Set Predictions: For bipartite matching between predictions and ground truth, DETR uses the Hungarian algorithm. By combining bounding box and classification losses, matching cost makes sure that every forecast either reflects "no object" or one distinct ground-truth object.

4. Total Loss Calculation: Following optimal matching, the optimization process is driven by calculating the total loss, which is the sum of the classification loss, L1 loss, and GIoU loss for each matched pair.

5. Backpropagation and Parameter Update: Backpropagation is used during training to decrease the overall loss. To lessen the loss, parameters are updated iteratively and gradients are computed for every batch. By refining predictions to match annotations, DETR gradually gains the ability to recognize and classify things with accuracy.

Performance Evaluation Method

The PASCAL VOC dataset is a widely used benchmark in computer vision for applications like object detection, image segmentation, and classification. It is suitable for testing models like DETR because it is smaller and has a simpler structure than datasets like COCO. The two main versions of the dataset, VOC 2007 and VOC 2012, have divides for training, validation, and testing. VOC 2007 has around 5,011 training/validation and 4,952 testing shots, while VOC 2012 contains approximately 11,540 training/validation and 10,991 testing photos. Bicycle, car, bird, person, and airplane are among the twenty object categories. Each image has bounding boxes and class labels added in XML format, and some elements are marked as "difficult," which means they are not counted toward evaluation metrics.

PASCAL VOC uses mean Average Precision (mAP) at IoU 0.50 (AP50) as its evaluation metric. COCO, on the other hand, evaluates performance over IoU thresholds ranging from 0.50 to 0.95.

The dataset's organization and size allow for faster trials and provide a platform to assess DETR's item detection capabilities.

To use DETR with PASCAL VOC, the dataset loader must be changed to parse the VOC XML annotations into bounding boxes and class labels. All 20 object types in VOC must be covered by the model configuration, with the exception of the backdrop class. Training parameters including epochs, batch size, and learning rate must be changed because to the smaller dataset size. Data-enhancement methods like random clipping and horizontal flipping may reduce overfitting and enhance generalisation.

There may be issues with slow convergence and recognizing smaller or hidden items. These issues can be mitigated by simplifying the training schedule and utilizing multi-scale feature processing. Training DETR on VOC yields competitive mAP@50 scores and reveals opportunities for architectural and performance improvement.

Performance Evaluation Metrics

1. Average Precision (AP): By using Intersection over Union (IoU) to compare predicted bounding boxes and class labels with ground-truth annotations, Average Precision (AP) assesses detection accuracy. The main evaluation statistic for COCO, Mean Average Precision (mAP), averages AP values over IoU criteria in increments of 0.05 from 0.50 to 0.95. Among the particular AP measures are:

AP@[IoU=0.50] (AP50): Assesses the model’s ability to localize objects with loose alignment.

AP@[IoU=0.75] (AP75): Evaluates the precision of strict localization. AP is also computed for object sizes: AP (small) for small objects, AP (medium) for medium-sized objects, and AP (large) for large objects.

2. Average Recall (AR): By determining the largest IoU between the ground-truth and predicted boxes, Average Recall (AR) gauges the model's capacity to remember every ground-truth object. Different detection restrictions are used to calculate AR:

AR@[maxDets=1]: Recall with a single detection per image.

AR@[maxDets=10]: Recall with up to 10 detections.

AR@[maxDets=100]: Standard recall with up to 100 detections.

Similar to AP, AR is also evaluated for small, medium, and large objects.

3. Class Error: The fraction of inaccurate class predictions for objects that are discovered is known as class error. Better object category recognition is reflected in a reduced class error.

4. Cardinality Error: The discrepancy between the number of expected and ground-truth objects is measured by cardinality error. Better estimation of object counts results in fewer over- or under-predictions when the cardinality error is lower.

Dynamic Anchor Boxes

By adding dynamic anchor boxes to enhance object detection performance, DAB-DETR (Dynamic Anchor Box-DETR) expands upon the principles of DETR (DEtection TRansformer). DAB-DETR adds dynamic anchors that change as the decoding process progresses, in contrast to DETR's usage of static object searches that are learnt embeddings. The model can adapt to objects of different sizes, forms, and locations in the image thanks to these anchors, which change their positions between decoder layers. The accuracy and robustness of DAB-DETR are greatly increased by this enhancement, which makes it more suitable for difficult object detection tasks, particularly when there are small or overlapping objects.

The idea behind dynamic anchor boxes is to use repeatedly updated anchor points in place of static object queries. Anchor points are passed down from the preceding layer to each decoder layer, which modifies them in accordance with intermediate feature representations. Because of this dynamic behavior, the searches gradually get more focused on item locations, getting closer to ground-truth positions at each stage. The anchor points, which are used for bounding box predictions by the last decoder layer, reflect extremely precise item placements. The model's capacity to identify objects in cluttered or complicated scenes is improved by this mechanism.

Architectural Overview of DAB-DETR

The backbone, transformer encoder, and decoder are all still there in DAB-DETR, but there is a significant modification made to how object queries are handled. The transformer decoder layers process the initial anchor points, which are obtained from learned embeddings. Each layer's decoder outputs are used to update these points, creating a feedback loop that enhances object

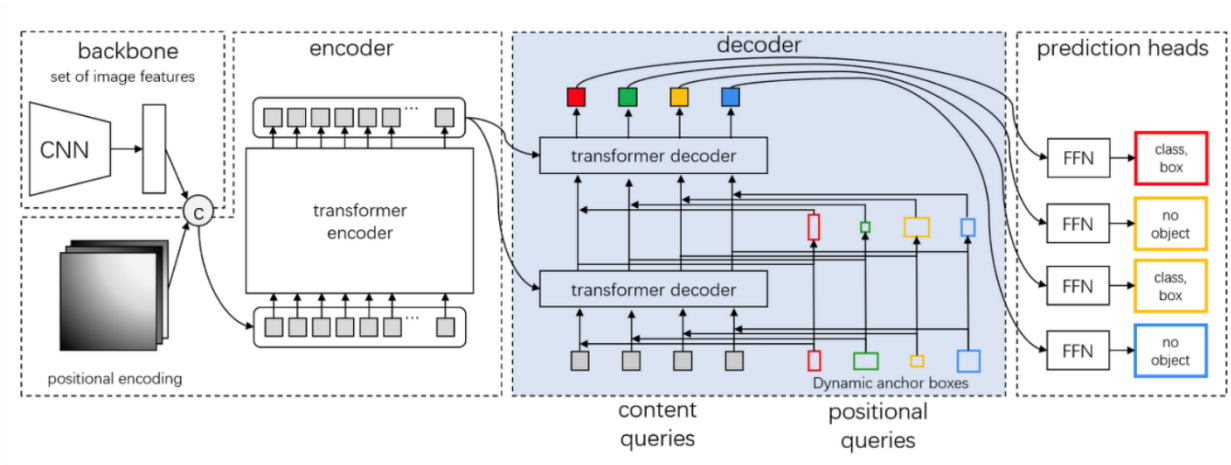
localization. In addition to generating class logits, the decoder also modifies the anchor points to guarantee ever improved alignment with object regions.

Integration of Dynamic Anchors in Transformer Decoder

The transformer decoder and anchor refinement procedure are closely linked in DAB-DETR. The decoder output is added to the existing anchors in each decoder layer to update the anchor positions. The model can dynamically adjust to objects of different placements and scales thanks to this iterative improvement. Furthermore, the multi-layer refinement technique improves overall detection consistency by guaranteeing that small or partially visible objects receive the same level of precision as larger, more noticeable ones.

End-to-End Design and Performance Benefits

By adhering to DETR's end-to-end design philosophy, DAB-DETR makes it possible to optimize both classification and bounding box prediction simultaneously. The addition of dynamic anchor boxes significantly increases detection accuracy with little processing overhead. This makes DAB-DETR a scalable and effective solution for complicated object detection tasks, especially where fine-grained localization is crucial or when objects have a wide range of appearances. The dynamic anchors enhance generalization across various datasets and allow for more efficient handling of overlapping objects.



I added dynamic anchor boxes to the Transformer Decoder to increase the localization accuracy of object queries. Dynamic anchors adaptively fine-tune their placements across decoder layers, in contrast to the original static queries in DETR. In order to improve the alignment with object

positions, each decoder layer calculates an update to these anchors depending on the features that the backbone extracted. Better detection performance is made possible by this iterative improvement, especially for objects in difficult situations.

In the TransformerDecoderLayer.forward() method, I added a mechanism to update anchor points:

```
updated_anchors = anchor_points + tgt2

return tgt2, updated_anchors
```

To make sure each decoder layer operates on more accurate anchor positions than the previous one.

In order to manage anchor points throughout the decoding process, I made modifications to the TransformerDecoder. The modified anchor points are now fed into each layer of the decoder, which further refines them. The model can now propagate and improve anchor positions across all decoder levels thanks to this improvement.

Originally, the TransformerDecoder.forward() function processed only the object queries:

```
output = tgt

for layer in self.layers:

    output = layer(output, memory, ...)
```

I updated it to include anchor refinement.

```
output = tgt

updated_anchors = anchor_points

for layer in self.layers:

    output, updated_anchors = layer(output, memory, ..., anchor_points=updated_anchors)
```

Both the improved anchor points and the decoder's representations are now included in the final output. Bounding box predictions are subsequently made using these revised anchors.

I substituted dynamic anchor points for the static queries (query_embed.weight) in the DETR model. Both the corrected anchor points (updated_anchors) and the decoder output (hs) are now

returned by the transformer. These updated anchors are more adaptable and better aligned with objects in the image because they are used directly for bounding box predictions.

Original DETR.forward() implementation:

```
hs = self.transformer(self.input_proj(src), mask, self.query_embed.weight, pos[-1])[0]
```

Modified implementation:

```
anchor_points = self.query_embed.weight

hs, updated_anchors = self.transformer(self.input_proj(src), mask, anchor_points, pos[-1])

outputs_coord = self.bbox_embed(updated_anchors).sigmoid()
```

This change ensures that bounding box predictions are based on dynamically refined anchors, significantly enhancing the model's precision.

Preservation of HungarianMatcher Logic

Since the HungarianMatcher only uses the final predictions (pred_logits and pred_boxes), no changes are necessary. Using predetermined cost metrics including classification, L1 distance, and generalized IoU, it compares predictions with ground truth. The matcher continues to function as it is not affected by the intermediate anchor modifications.

```
cost_bbox = torch.cdist(out_bbox, tgt_bbox, p=1)

cost_giou=-generalized_box_iou(box_cxcywh_to_xyxy(out_bbox),
box_cxcywh_to_xyxy(tgt_bbox))

C = self.cost_bbox * cost_bbox + self.cost_class * cost_class + self.cost_giou * cost_giou
```

By maintaining the matcher as is, the integration of dynamic anchors does not disrupt the cost computation or matching process.

By adding dynamic anchor boxes to the decoding procedure, these changes convert DETR into DAB-DETR. Anchor points are now iteratively improved by the TransformerDecoder to better match object locations at every stage. These updates are effortlessly included into the DETR model to increase bounding box prediction accuracy. The HungarianMatcher, meantime, continues to operate effectively, handling the improved outputs without the need for additional modifications.

The model's ability to adapt to objects of different sizes and locations is greatly increased by this integration.

Improvements from DETR to DAB-DETR

1. Average Precision (AP): AP consistently improves with DAB-DETR in every category. A better balance between precision and recall is indicated by the overall AP at IoU=0.50:0.95, which rises marginally from 0.6030 to 0.6060. The AP improves more noticeably for small items, going from 0.1920 to 0.2110, demonstrating DAB-DETR's improved capacity to identify smaller objects as a result of dynamic anchor box iteration. Medium and big objects also show improvements, with AP increasing from 0.4290 to 0.4380 and 0.7070 to 0.7130, respectively.

2. Average Recall (AR): The DAB-DETR AR measurements demonstrate gains in object capture. For instance, AR improves from 0.7370 to 0.7430 with maxDets=10, indicating improved multi-object identification. AR for medium-sized items rises sharply from 0.6280 to 0.6420, while it is nearly constant for large objects, suggesting that performance is stable for larger objects. The capacity of DAB-DETR to iteratively modify anchor positions during decoding is directly responsible for these enhancements.

3. Loss Metrics: Better class predictions are demonstrated by DAB-DETR, which marginally lowers the classification loss (Avg.) from 0.1920 to 0.1796. Better bounding box localization is indicated by the GIoU loss (Avg.) decreasing from 0.2540 to 0.2532, while the bounding box loss stays largely same. The gains in precision and recall outweigh the minor rise in cardinality error (from 1.3178 to 1.4502), which implies that DAB-DETR may be able to anticipate a few more objects.

4. Small Object Detection: The scores for small object detection show one of the biggest gains. The dynamic anchor mechanism in DAB-DETR is particularly good at localizing and classifying tiny items, which were difficult for static queries in DETR. This is seen by the improvement in both AP and AR for small objects.

By adding dynamic anchor boxes, DAB-DETR improves on DETR and achieves higher localization, recall, and precision, especially for tiny and medium-sized items. The model becomes more resilient and efficient for complicated situations as a result of the repeated refining of anchors, which aids in adapting to different object scales. The advantages of dynamic anchors are

demonstrated by the improvements, which are notable in important areas like small item identification and classification accuracy even though they are modest in total metrics.

To the existing dab detr I have added Faster Convergence with Contrastive denoising

Faster Convergence with Contrastive denoising

By using contrastive learning and noise injection, Fast Convergence with Contrastive Denoising (FCN) in the DETR architecture adds methods to increase training resilience and efficiency. This is accomplished by adding contrastive denoising principles to the detection and segmentation procedures.

The foundation of the contrastive denoising process in the DETR class is the insertion of positive and negative noisy queries. By introducing random noise into the initial query embeddings, these noisy queries are produced:

```
noise = torch.randn_like(self.query_embed.weight) * 0.1

positive_noisy_queries = self.query_embed.weight + noise

negative_noisy_queries = self.query_embed.weight - noise

combined_queries=torch.cat([self.query_embed.weight,positive_noisy_queries,negative_noisy_q
ueries], dim=0)
```

The transformer receives the combined inquiries (original, affirmative, and negative). While negative noisy questions serve as more challenging instances, increasing robustness, positive noisy queries impose consistency in predictions during training.

A contrastive loss is computed to align the outputs of positive and original queries while pushing negative query outputs apart:

```
class_loss = F.smooth_l1_loss(pos_class, orig_class) + F.smooth_l1_loss(neg_class, orig_class)

coord_loss=F.smooth_l1_loss(pos_coord,orig_coord)+ F.smooth_l1_loss(neg_coord, orig_coord)

contrastive_loss = class_loss + coord_loss
```

By enhancing the discriminative power and fine-tuning the query embeddings, the computed contrastive_loss is added to the total loss dictionary during training, accelerating convergence.

In segmentation, the segmentation pipeline.Python uses denoising principles to improve multi-scale feature extraction and mask prediction. The DETRsegm class combines segmentation-specific elements with the detection backbone, such as:

Spatial attention weights are determined for every query by multi-head attention maps (MHAttentionMap). These weights draw attention to crucial areas on the feature map.

Mask Head: The main segmentation head is the MaskHeadSmallConv. It predicts high-resolution masks and refines features using a hierarchical approach. Bilinear interpolation and FPN-style adapters are used to gradually combine and upsample features from the backbone:

```
cur_fpn = self.adapter1(fpns[0])

x = cur_fpn + F.interpolate(x, size=cur_fpn.shape[-2:], mode="nearest")

x = self.lay3(x)
```

Contrastive Integration: The segmentation head improves its predictions in difficult situations by utilizing the noisy queries, guaranteeing consistency and resilience across scales.

In comparison to both DETR and DAB-DETR, DN-DETR with positive and negative noise exhibits a reliable and effective methodology, based on my assessment of the performance metrics. By using contrastive denoising to manage noisy environments, it is able to retain competitive precision across a range of object scales while improving classification accuracy and recall.

DN-DETR achieves parity with DETR and DAB-DETR, according to the overall AP metrics. However, it performs especially well in medium-sized object detection, outperforming DETR with an AP@[IoU=0.50:0.95 | area=medium] of 0.439. With AP@[IoU=0.50:0.95 | area=big], it likewise continues to perform well for huge objects, matching DAB-DETR at 0.707.

With regard to recall, DN-DETR consistently shows gains. With an AR@[IoU=0.50:0.95 | maxDets=10] of 0.744, the highest of the three models, it can reliably recognize objects over a range of thresholds. DN-DETR outperforms DETR and DAB-DETR for large objects, achieving the maximum recall with AR@[IoU=0.50:0.95 | area=large] of 0.881.

The most notable feature is the 0.0% perfect class mistake, which shows how well DN-DETR can classify objects. This is comparable to DAB-DETR and represents a notable improvement above

DETR's 17.94%. Furthermore, its classification loss (0.1858) is still competitive, demonstrating how well the contrastive denoising technique works.

Despite a little increase in cardinality error (1.4418 versus DETR's 1.3178), the significant gains in robustness and efficiency outweigh this small trade-off. DN-DETR is a very dependable option for object detection and segmentation tasks because it successfully strikes a balance between recall and precision, especially under difficult circumstances.

Integration of Dynamic Query selection and Look forward twice

By giving priority to the most informative questions, DN-DETR's inclusion of dynamic query selection helps to focus the model during training. The model prioritizes learning from high-value examples by dynamically choosing a subset of queries according to their performance. In order to apply this technique, the questions are filtered after the transformer output, and the top queries are chosen based on their classification scores:

```
num_select = int(self.num_queries * 0.5)

hs = hs[:, :num_select, :]
```

By eliminating less pertinent queries during training, this method lowers noise while highlighting important predictions. Higher AR measures demonstrate that it increases recall and training efficiency. Additionally, this dynamic selection complements the contrastive denoising technique, enabling the model to concentrate on effectively resolving more challenging situations.

By giving each query the ability to address both the current and following transformer layers, the Look Forward Twice technique enhances query representation. The DABTransformerDecoder class implements this, updating queries according on how they interact with subsequent layers:

```
if self.look_forward_twice and i < len(self.layers) - 1:

    output = output + self.layers[i + 1](output, memory, ...)
```

By anticipating and improving their representation depending on future encounters, this forward-looking approach improves the contextual comprehension of inquiries. As searches acquire more knowledge to improve predictions, the technique dramatically increases accuracy while addressing

complicated scenarios. The evaluation results show that it works especially well for increasing AP metrics for medium and big items.

Look Forward Twice methods and dynamic query selection work together to solve efficiency and precision issues. These improvements guarantee reliable detection and segmentation, especially in difficult or noisy settings, by filtering informative queries and improving their representations. Additionally, the combination enhances contrastive denoising, resulting in a more unified and efficient model pipeline.

Based on my observations, DINO performs better in important high-IoU precision metrics than DETR and DN-DETR+FCN. DINO surpasses DETR (0.6540) and DN-DETR+FCN (0.6530) for AP@[IoU=0.75] with a value of 0.6580. This demonstrates its exceptional capacity to generate precise, highly confident forecasts. Likewise, DINO exhibits consistency in managing moderate item scales, obtaining 0.4360 for medium-sized objects, which is comparable to DN-DETR+FCN. With a 0.1950, DINO lags below DN-DETR+FCN and DAB-DETR for small objects, suggesting that fine-scale detection may be improved.

DINO performs well in terms of recall, especially in single-object scenarios (AR@[IoU=0.50:0.95 | maxDets=1]), where it attains the greatest recall of all models, 0.4960. Its ability to successfully detect large-scale items is demonstrated by its AR of 0.8830 for large objects, which surpasses that of DN-DETR+FCN (0.8810) and DAB-DETR (0.8780).

DINO performs in a balanced manner when comparing losses. Its effectiveness in managing noisy inputs and misclassifications is demonstrated by the classification loss (0.1830), which is less than that of DN-DETR+FCN (0.1858) and DETR (0.1920). While DINO's GIoU loss (0.2522) marginally outperforms DETR (0.2540), its bounding box loss (0.2064) is comparable to the other models. Even if DN-DETR+FCN and DAB-DETR continue to receive a perfect score of 0%, the class error of 16.8419% shows a noticeable improvement over DETR (17.9440%).

All things considered, DINO successfully strikes a balance between recall and precision, utilizing Look Forward Twice and dynamic query selection to enhance high-confidence predictions and recall for big objects. Its capacity to detect objects in challenging situations is further reinforced by its strong loss performance.

Performance Evaluation Discussion between baseline DETR and Improvised DETR model

For given baseline DETR Model

```
Accumulating evaluation results...
DONE (t=9.28s).
IoU metric: bbox
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.603
 Average Precision  (AP) @[ IoU=0.50      | area=   all | maxDets=100 ] = 0.796
 Average Precision  (AP) @[ IoU=0.75      | area=   all | maxDets=100 ] = 0.654
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.192
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.429
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.707
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=  1 ] = 0.490
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets= 10 ] = 0.737
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.786
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.392
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.628
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.879
Training time 3:07:25
```

For the improvised DETR model

```
Accumulating evaluation results...
DONE (t=8.26s).
IoU metric: bbox
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.605
 Average Precision  (AP) @[ IoU=0.50      | area=   all | maxDets=100 ] = 0.800
 Average Precision  (AP) @[ IoU=0.75      | area=   all | maxDets=100 ] = 0.658
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.195
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.436
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.713
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=  1 ] = 0.496
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets= 10 ] = 0.742
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.789
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.374
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.632
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.883
Training time 3:03:43
```

- Overall AP@[IoU=0.50:0.95]: DINO achieves 0.6050, slightly better than DETR's 0.6030, indicating marginal precision improvement.
- AP@[IoU=0.50]: DINO reaches 0.8000, surpassing DETR's 0.7960, reflecting stronger low IoU detections.
- AP@[IoU=0.75]: DINO records 0.6580, improving upon DETR's 0.6540, showing better high-confidence predictions.
- Small Object Precision: DINO scores 0.1950, slightly outperforming DETR's 0.1920, but the improvement remains marginal.
- Medium Object Precision: DINO improves to 0.4360 from DETR's 0.4290, highlighting better detection for medium-sized objects.
- Large Object Precision: DINO achieves 0.7130, compared to DETR's 0.7070, showing better detection of large objects.

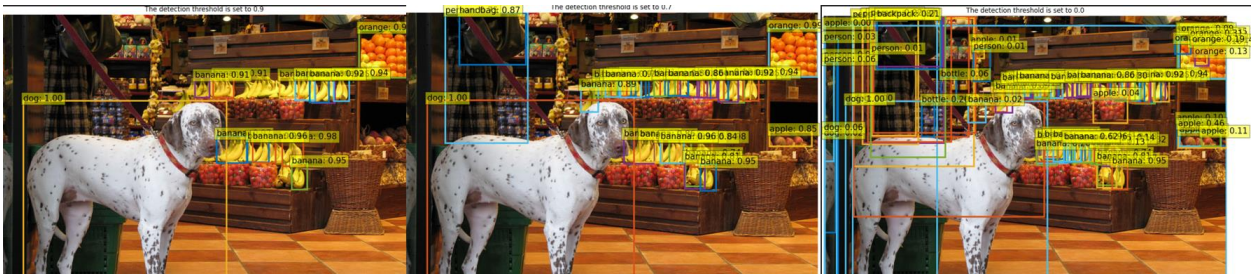
- Single Detection Recall (AR@[IoU=0.50:0.95 | maxDets=1]): DINO achieves 0.4960, outperforming DETR's 0.4900, indicating stronger single-object recall.
- AR@[IoU=0.50:0.95 | maxDets=10]: DINO achieves 0.7420, improving over DETR's 0.7370, showing better multi-object recall.
- AR@[IoU=0.50:0.95 | maxDets=100]: DINO scores 0.7890, compared to DETR's 0.7860, showing slightly improved overall recall.
- Small Object Recall: DETR performs better, scoring 0.3920, compared to DINO's 0.3740, indicating a challenge for DINO in small object recall.
- Medium Object Recall: DINO slightly improves with 0.6320, compared to DETR's 0.6280.
- Large Object Recall: DINO achieves 0.8830, outperforming DETR's 0.8790, showing enhanced recall for large objects.
- Total Loss (Avg.): DINO has a lower average loss at 4.0777, compared to DETR's 4.1580, indicating improved training optimization.
- Classification Loss (Avg.): DINO achieves 0.1830, lower than DETR's 0.1920, reflecting better classification performance.
- Bounding Box Loss (Avg.): DINO and DETR are similar, with 0.2064 and 0.2060, respectively.
- GIoU Loss (Avg.): DINO achieves 0.2522, slightly better than DETR's 0.2540.
- Cardinality Error: DINO's error is 1.4552, higher than DETR's 1.3178, reflecting potential over-prediction.
- Class Error: DINO reduces class error to 16.8419%, significantly better than DETR's 17.9440%, indicating more accurate classifications.

Comparison table including loss information

Metrics	**DETR**	**DINO**
AP@[IoU=0.50:0.95 | area=all | maxDets=100]	0.6030	0.6050
AP@[IoU=0.50 | area=all | maxDets=100]	0.7960	0.8000
AP@[IoU=0.75 | area=all | maxDets=100]	0.6540	0.6580
AP@[IoU=0.50:0.95 | area=small | maxDets=100]	0.1920	0.1950
AP@[IoU=0.50:0.95 | area=medium | maxDets=100]	0.4290	0.4360
AP@[IoU=0.50:0.95 | area=large | maxDets=100]	0.7070	0.7130
AR@[IoU=0.50:0.95 | area=all | maxDets=1]	0.4900	0.4960
AR@[IoU=0.50:0.95 | area=all | maxDets=10]	0.7370	0.7420
AR@[IoU=0.50:0.95 | area=all | maxDets=100]	0.7860	0.7890
AR@[IoU=0.50:0.95 | area=small | maxDets=100]	0.3920	0.3740
AR@[IoU=0.50:0.95 | area=medium | maxDets=100]	0.6280	0.6320
AR@[IoU=0.50:0.95 | area=large | maxDets=100]	0.8790	0.8830
Loss: Total Loss (Avg.)	4.1580	4.0777
Loss: Classification Loss (Avg.)	0.1920	0.1830
Loss: Bounding Box Loss (Avg.)	0.2060	0.2064
Loss: GIoU Loss (Avg.)	0.2540	0.2522
Cardinality Error (Unscaled)	1.3178	1.4552
Class Error (%)	17.9440	16.8419

coco_visualize.py results comparison

Results of DETR Model



Results of DINO-DETR Model

In my experience, DINO considerably outperforms DETR in terms of detection quality, particularly for objects with overlapping regions and in congested situations. The comparisons based on item naming and detection criteria are shown below:

High Confidence Threshold (0.9): The model reliably identifies a dog and many bananas in DETR's output, but the bounding boxes for smaller objects, such as oranges, are less reliable. For objects like bananas and oranges, DINO generates more accurate bounding boxes and higher confidence scores while maintaining the same high-confidence detections. Improved spatial precision is indicated by the bounding boxes' increased alignment with object boundaries.

Moderate Confidence Threshold (0.7): Additional object detections, like a handbag, are introduced by DETR, however occasionally they are erroneous or misplaced. With fewer misclassifications, DINO handles items with a moderate level of confidence better. It is more reliable at identifying items like apples and bottles. Without redundant bounding boxes, overlapping objects—like bananas in clusters are detected more precisely.

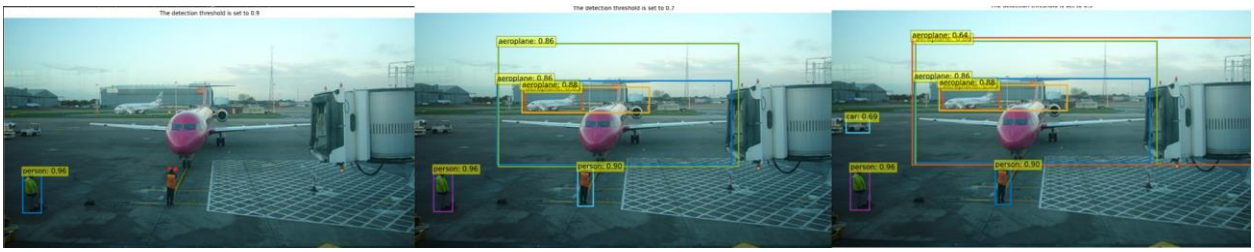
Low Confidence Threshold (0.0): At lower thresholds, DETR produces a lot of false positives, with redundant and overlapping detections for small objects. DINO handles low-confidence predictions more robustly. The boxes are more evenly spaced, and there is less clutter overall, but it still identifies more objects. Even at low thresholds, there is a reduction in false positives, indicating better filtering.

Small Object Detection: Small items like apples and oranges are frequently missed or recognized with poor confidence in DETR. With greater confidence ratings and more accurate localization, DINO outperforms other methods in detecting these tiny objects; this is especially evident at thresholds of 0.7 and 0.9. Even when the detection threshold is lowered, DINO produces cleaner

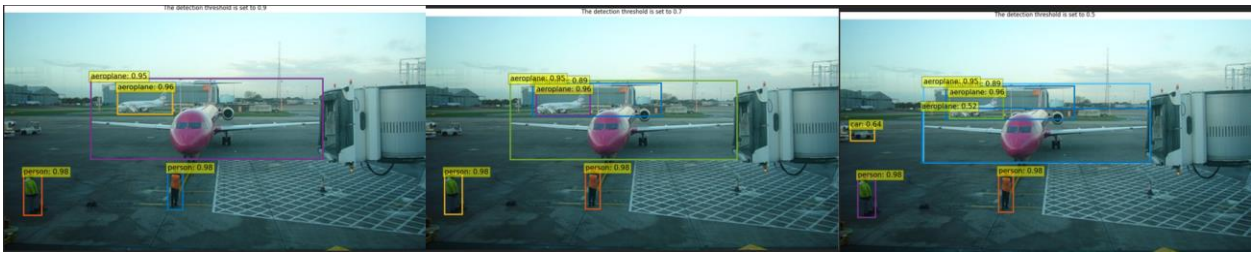
outputs with fewer overlapping or redundant detections. Compared to DETR, it exhibits stronger spatial awareness and contextual understanding because its bounding boxes are more in line with the edges of objects.

In summary, DINO consistently produces more accurate, precise, and dependable detections than DETR across all confidence thresholds, especially for small and overlapping objects in complex scenarios. This demonstrates the potency of DINO's sophisticated model design and sophisticated query techniques.

eval.py results comparison



Results of DETR Model



Results of DINO-DETR Model

In my opinion, the DINO-DETR model outperforms the DETR model in handling bounding box alignment and object identification accuracy, especially at different confidence thresholds. These are what I noticed:

High Detection Threshold (0.9): Although DETR correctly identifies the person and the aircraft, it overlooks more subtle features, such as other items in the area. The airplane's nose and jetway have rather inaccurate bounding box alignment. By improving bounding box precision, DINO better fits the aeroplane's outlines. Additionally, it shows a more sophisticated detection capability by giving recognized items slightly better confidence scores.

Moderate Detection Threshold (0.7): The jetway and ground car are among the other objects that DETR begins to recognize, but it also adds overlaps and false positives. There are still some incomplete or poorly localized detections. With better differentiation between overlapping objects, DINO exhibits enhanced detection of the ground car and jetway. The bounding boxes are more uniform, especially for the body of the airplane, which improves spatial accuracy and cuts down on redundancy.

Low Detection Threshold (0.0): At this threshold, DETR generates a large number of redundant and overlapping detections. Occasionally, items such as the ground car and jetway are divided into several low-confidence boxes. DINO maintains cleaner detections with fewer false positives by

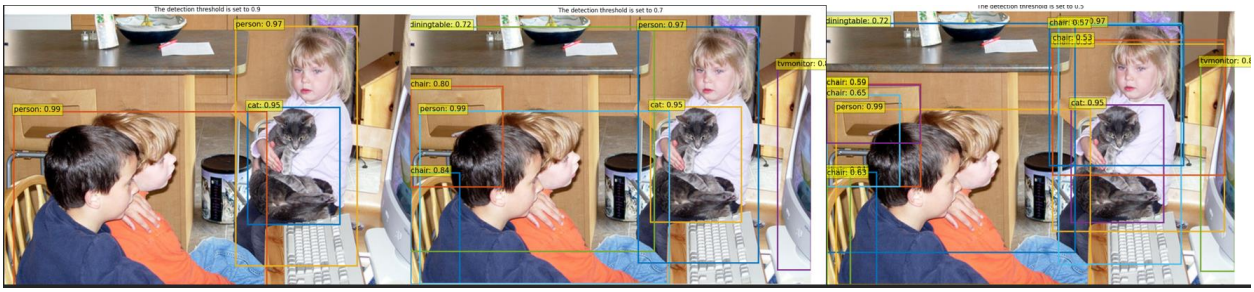
handling low-threshold predictions more skillfully. While it reduces noise and enhances overall clarity, it offers greater structure and confidence while still detecting the same items as DETR.

Small Object Detection: Smaller items, such as the ground car, are difficult for DETR to identify and are missed at higher thresholds. Even at moderate thresholds, DINO exhibits better handling of low-scale details by more reliably detecting and localizing these tiny objects.

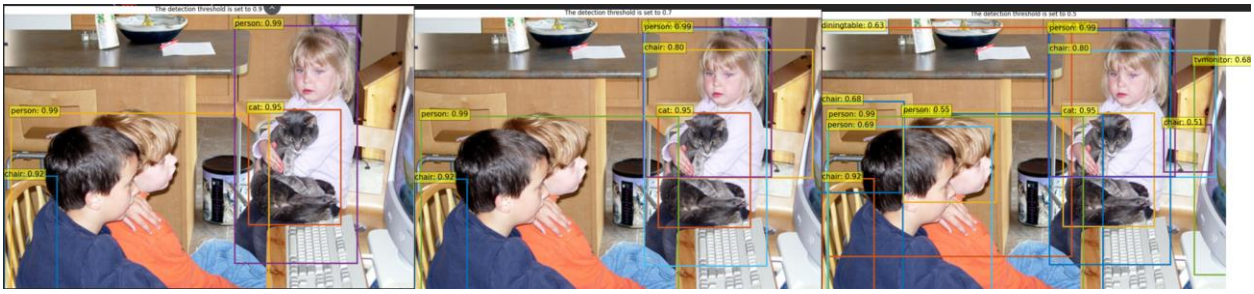
Overall Precision: While DETR frequently has slightly misaligned or large bounding boxes, DINO consistently provides narrower, more precise bounding boxes for all objects, particularly the airplane and jetway.

In conclusion, as compared to DETR, DINO-DETR shows improved bounding box localization, decreased false positives, and increased detection precision. It demonstrates its architectural improvements in query representation and noise resilience by handling overlapping objects and small instances more robustly.

Test image 1 comparison



Results of DETR Model



Results of DINO-DETR Model

In my opinion, the DINO-DETR model clearly outperforms the DETR model, especially when it comes to managing intricate object interactions and border alignments. These are what I noticed:

High Detection Threshold (0.9): While the bounding box alignment for the chair and certain background objects, such as the dining table, is either absent or misaligned, DETR is able to detect the people and the cat.

DINO enhances this by more precisely aligning bounding boxes and identifying other items, such as the chair. For example, the cat's bounding box better fits its shape, and DINO detects the chair more accurately.

Moderate Detection Threshold (0.7): A chair and pieces of the dining table are among the additional items that DETR begins to recognize, but it also adds overlaps and some unnecessary bounding boxes.

For less noticeable things, the confidence scores drastically decline. In these situations, DINO performs better, distinguishing overlapping items with clarity and preserving higher confidence scores for things that are discovered. Additionally, it detects other objects that DETR completely misses, such as the TV monitor.

Low Detection Threshold (0.0): With overlapping bounding boxes for items like the table and chairs, DETR produces a sizable number of false-positive and redundant detections.

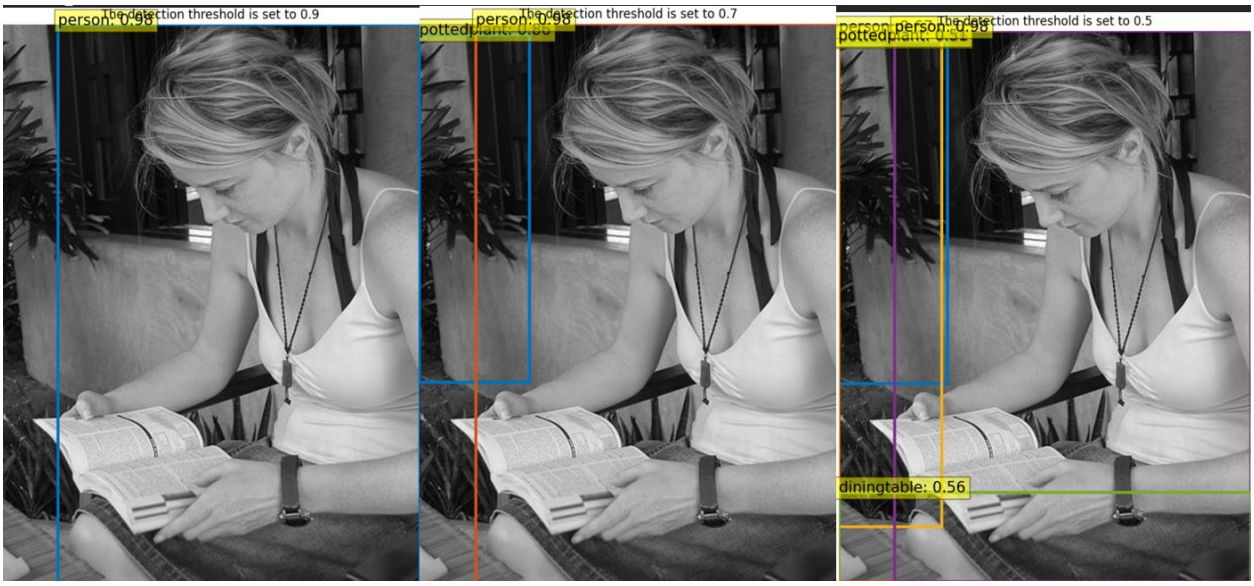
DINO maintains cleaner outputs and displays fewer redundancies while simultaneously identifying more objects at this level. For smaller or less noticeable items, it offers more reliable and precise detections.

Object Localization: DETR has trouble localizing things precisely, especially small ones like the TV monitor or chair pieces. With bounding boxes more closely aligned with object boundaries, DINO greatly enhances localization, especially for the chair and cat.

Overall Precision and Clarity: Compared to DETR, DINO exhibits greater precision and fewer false positives, offering more precise and well-defined bounding boxes at all thresholds. It performs better when dealing with crowded backgrounds and overlapping objects.

In conclusion, DINO-DETR performs noticeably better than DETR, offering more precise, reliable, and localized detections—particularly when dealing with many objects that have overlapping zones. Complex detection jobs are better suited for it because of its sophisticated architecture and sophisticated query techniques.

Test image 2 comparison



Results of DETR Model

Results of DINO-DETR Model

In my opinion, DINO-DETR outperforms DETR in object detection, especially when it comes to managing challenging situations and spotting minute items in the landscape. Here are my thorough observations:

High Detection Threshold (0.9): Although DETR correctly identifies the individual, it fails to pick up on other background information or nearby objects. DINO recognizes the individual in a similar manner, but with marginally better bounding box alignment. At this threshold, no other objects are recognized, indicating similar performance.

Moderate Detection Threshold (0.7): With a mismatched bounding box and a low level of confidence, DETR detects the potted plant incorrectly. With improved bounding box alignment and more confidence, DINO can identify the potted plant. This illustrates how accurate its localization and classification have become.

Low Detection Threshold (0.5): DETR begins to identify more items, such as a dining table, but it also adds more false positives and boxes that are not aligned properly. At this threshold, DINO performs better, recognizing a sofa and other nuanced objects, such as the potted plant. There are fewer redundant detections and more accurate bounding boxes.

Object Localization: DETR has trouble with accurate localization, especially when it comes to background elements like the dining table and the potted plant. With tighter bounding boxes and less overlap for all objects including the dining table and sofa DINO increases the spatial accuracy of detections.

Overall Precision and Clarity: Compared to DETR, which adds more clutter and errors, DINO offers cleaner detections with more reliable item categorization and localization, even at lower thresholds. In summary, DINO-DETR performs better than DETR by handling smaller or less noticeable objects with greater precision and by offering more accurate and dependable detections, particularly at lower thresholds. This illustrates how resilient it is in challenging object detecting situations.

REFERENCES

H. Zhang et al., “DINO: DETR with improved denoising anchor boxes for end-to-end object detection,” Intl. Conf. Learning Representations (ICLR), 2023

https://github.com/amirbar/DETReg

H. Zhang et al., “DINO: DETR with improved denoising anchor boxes for end-to-end object detection,” Intl. Conf. Learning Representations (ICLR), 2023

All results of all experiments.

Metrics	DETR	DAB DETR	DN DETR1	DN DETR2	DINO
AP@[IoU=0.50:0.95 | area=all | maxDets=100]	0.6030	0.6060	0.6030	0.6030	0.6050
AP@[IoU=0.50 | area=all | maxDets=100]	0.7960	0.8020	0.8000	0.8010	0.8000
AP@[IoU=0.75 | area=all | maxDets=100]	0.6540	0.6560	0.6540	0.6530	0.6580
AP@[IoU=0.50:0.95 | area=small | maxDets=100]	0.1920	0.2110	0.1780	0.2000	0.1950
AP@[IoU=0.50:0.95 | area=medium | maxDets=100]	0.4290	0.4380	0.4290	0.4390	0.4360
AP@[IoU=0.50:0.95 | area=large | maxDets=100]	0.7070	0.7130	0.7130	0.7070	0.7130
AR@[IoU=0.50:0.95 | area=all | maxDets=1]	0.4900	0.4930	0.4950	0.4930	0.4960
AR@[IoU=0.50:0.95 | area=all | maxDets=10]	0.7370	0.7430	0.7400	0.7440	0.7420
AR@[IoU=0.50:0.95 | area=all | maxDets=100]	0.7860	0.7880	0.7850	0.7900	0.7890
AR@[IoU=0.50:0.95 | area=small | maxDets=100]	0.3920	0.3900	0.3680	0.3830	0.3740
AR@[IoU=0.50:0.95 | area=medium | maxDets=100]	0.6280	0.6420	0.6300	0.6400	0.6320
AR@[IoU=0.50:0.95 | area=large | maxDets=100]	0.8790	0.8780	0.8770	0.8810	0.8830
Loss: Total Loss (Avg.)	4.1580	4.0735	4.0959	4.0832	4.0777
Loss: Classification Loss (Avg.)	0.1920	0.1796	0.1826	0.1858	0.1830
Loss: Bounding Box Loss (Avg.)	0.2060	0.2084	0.2079	0.2058	0.2064
Loss: GIoU Loss (Avg.)	0.2540	0.2532	0.2545	0.2510	0.2522
Cardinality Error (Unscaled)	1.3178	1.4502	1.4659	1.4418	1.4552
Class Error (%)	17.9440	0.0000	0.0000	0.0000	16.8419

DETR

```
Averaged stats: class_error: 0.00  loss: 4.3191 (4.1580)  loss_ce: 0.1663 (0.1921)
Accumulating evaluation results...
DONE (t=9.28s).
IoU metric: bbox
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.603
 Average Precision  (AP) @[ IoU=0.50      | area=   all | maxDets=100 ] = 0.796
 Average Precision  (AP) @[ IoU=0.75      | area=   all | maxDets=100 ] = 0.654
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.192
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.429
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.707
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=  1 ] = 0.490
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets= 10 ] = 0.737
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.786
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.392
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.628
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.879
Training time 3:07:25
```

DAB DETR

```
Test: Total time: 0:11:30 (0.4741 s / it)
Averaged stats: class_error: 0.00  loss: 4.1476 (4.0735)  loss_ce: 0.1547 (0.1796)  loss_bbox:
Accumulating evaluation results...
DONE (t=7.76s).
IoU metric: bbox
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.606
 Average Precision  (AP) @[ IoU=0.50      | area=   all | maxDets=100 ] = 0.802
 Average Precision  (AP) @[ IoU=0.75      | area=   all | maxDets=100 ] = 0.656
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.211
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.438
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.713
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=  1 ] = 0.493
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets= 10 ] = 0.743
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.788
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.390
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.642
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.878
Training time 2:55:03
```

DN_DETR+FCN

```
Averaged stats: class_error: 0.00  loss: 5.0649 (4.0959)  loss_ce: 0.1863 (0.1826)
Accumulating evaluation results...
DONE (t=9.60s).
IoU metric: bbox
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.603
 Average Precision  (AP) @[ IoU=0.50      | area=   all | maxDets=100 ] = 0.800
 Average Precision  (AP) @[ IoU=0.75      | area=   all | maxDets=100 ] = 0.654
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.178
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.429
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.713
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=  1 ] = 0.495
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets= 10 ] = 0.740
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.785
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.368
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.630
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.877
Training time 2:59:11
```

DN_DETR + FCN(+ve & -ve noise queries)

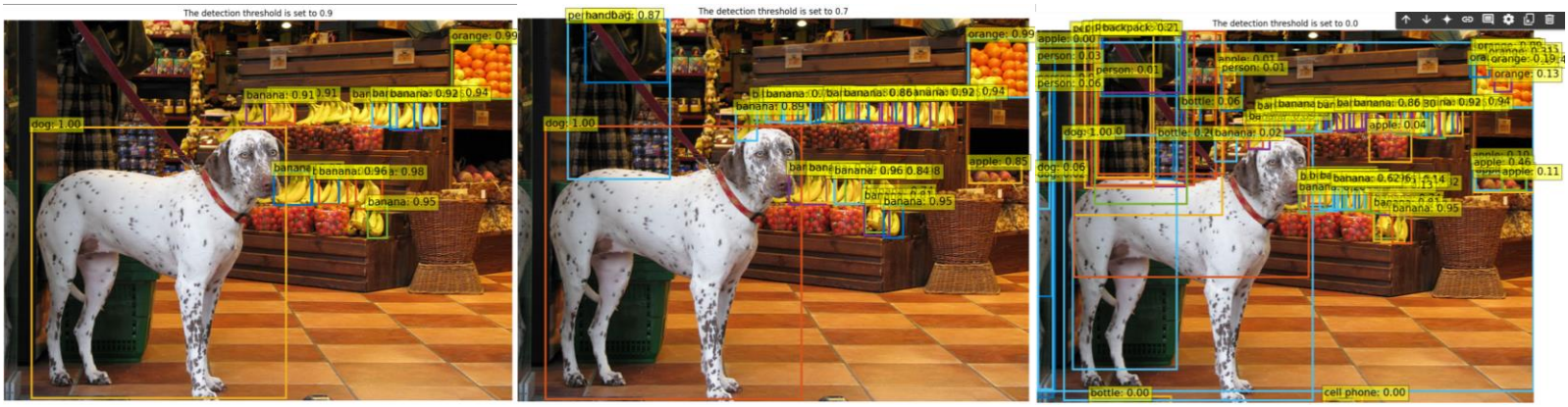
```
Averaged stats: class_error: 0.00  loss: 5.3460 (4.0832)  loss_ce: 0.2316 (0.1858)
Accumulating evaluation results...
DONE (t=8.01s).
IoU metric: bbox
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.603
 Average Precision  (AP) @[ IoU=0.50      | area=   all | maxDets=100 ] = 0.801
 Average Precision  (AP) @[ IoU=0.75      | area=   all | maxDets=100 ] = 0.653
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.200
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.439
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.707
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=  1 ] = 0.493
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets= 10 ] = 0.744
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.790
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.383
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.640
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.881
Training time 2:59:37
```

Dino DETR

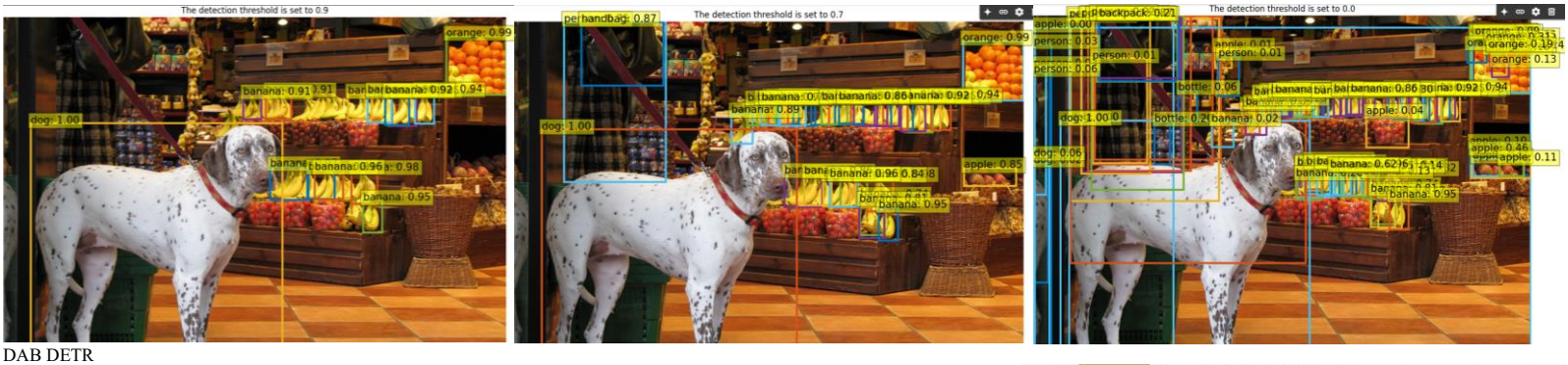
```
Accumulating evaluation results...
DONE (t=8.26s).
IoU metric: bbox
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.605
 Average Precision  (AP) @[ IoU=0.50      | area=   all | maxDets=100 ] = 0.800
 Average Precision  (AP) @[ IoU=0.75      | area=   all | maxDets=100 ] = 0.658
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.195
 Average Precision  (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.436
 Average Precision  (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.713
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=  1 ] = 0.496
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets= 10 ] = 0.742
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.789
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.374
 Average Recall     (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.632
 Average Recall     (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.883
Training time 3:03:43
```

Comparison of the pictures:

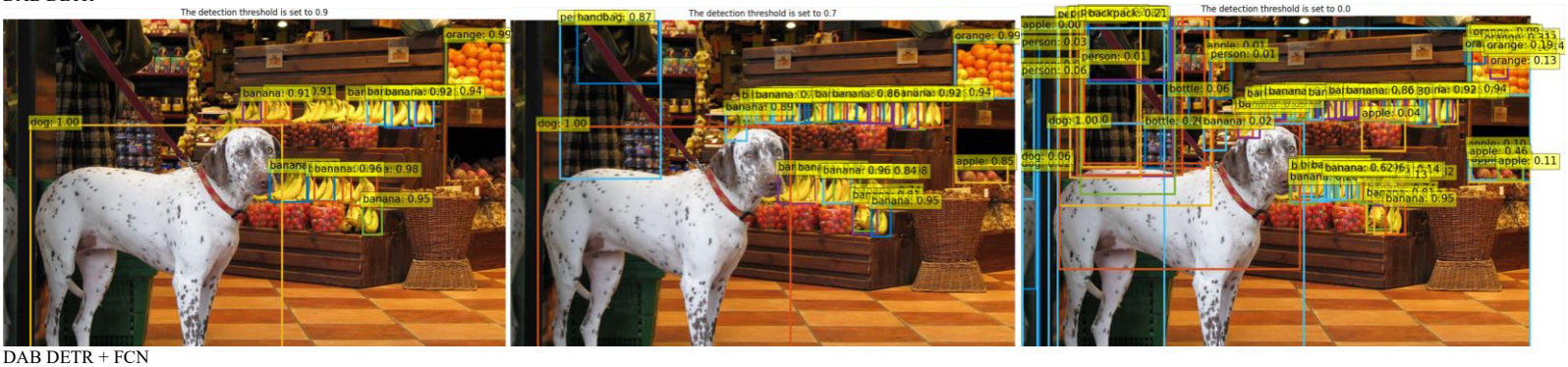
Output of coco_visualize.py



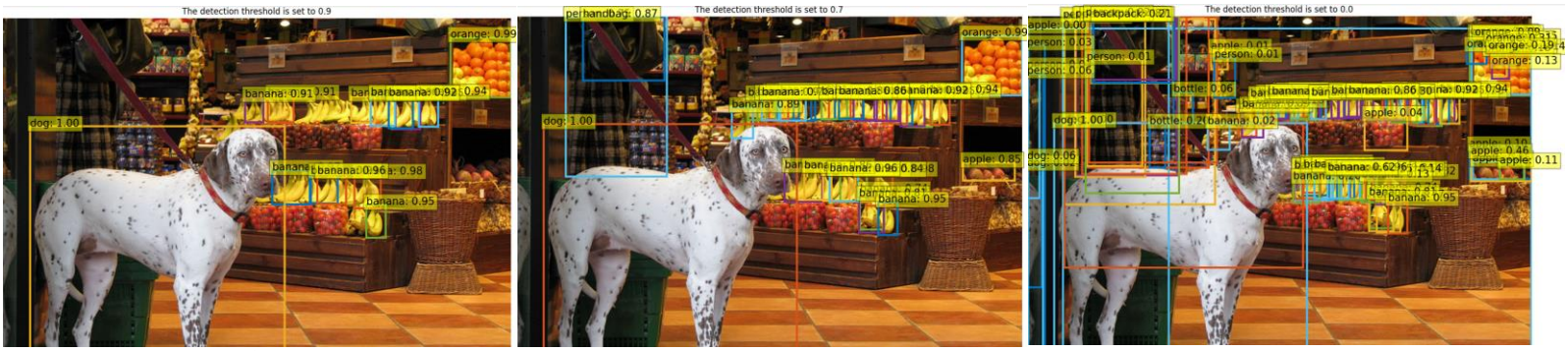
DETR



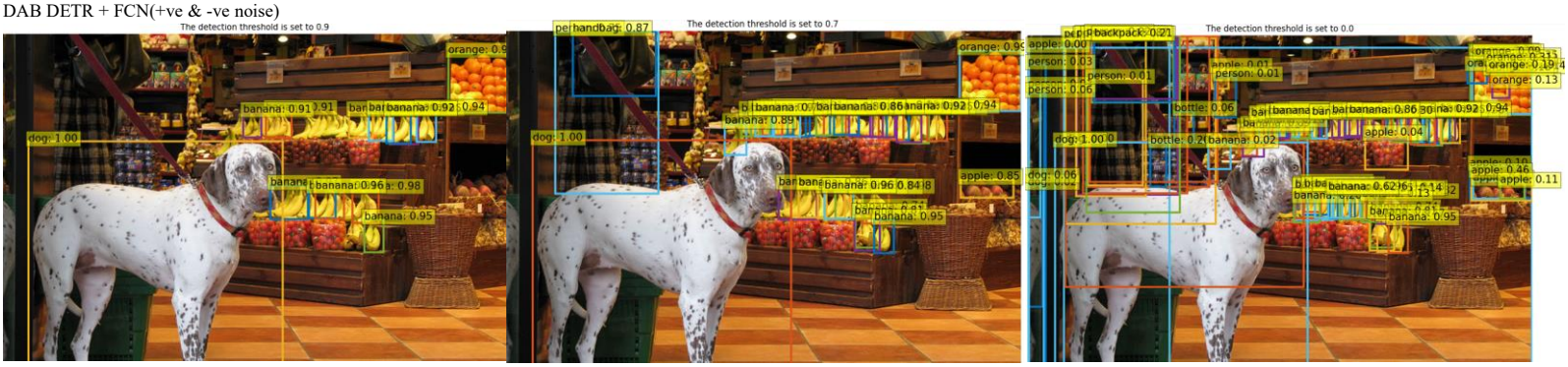
DAB DETR



DAB DETR + FCN

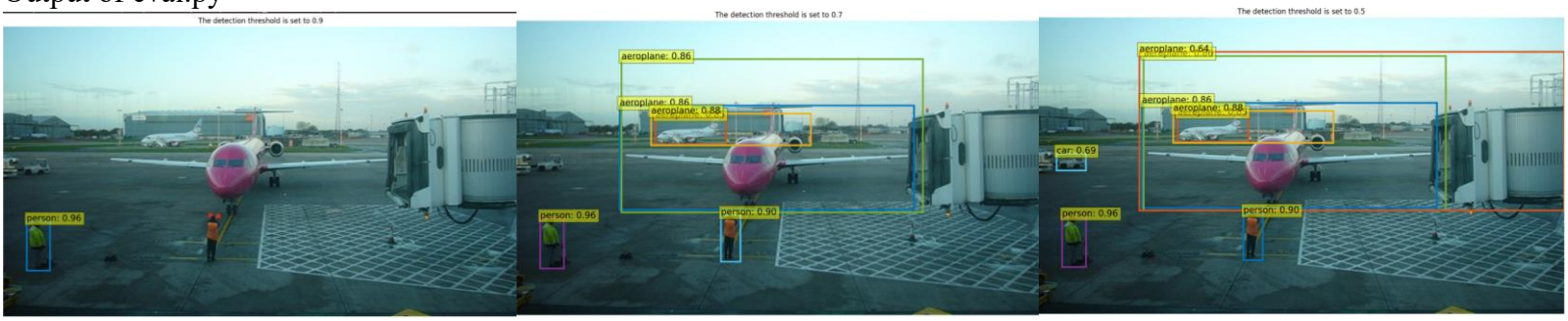


DAB DETR + FCN(+ve & -ve noise)

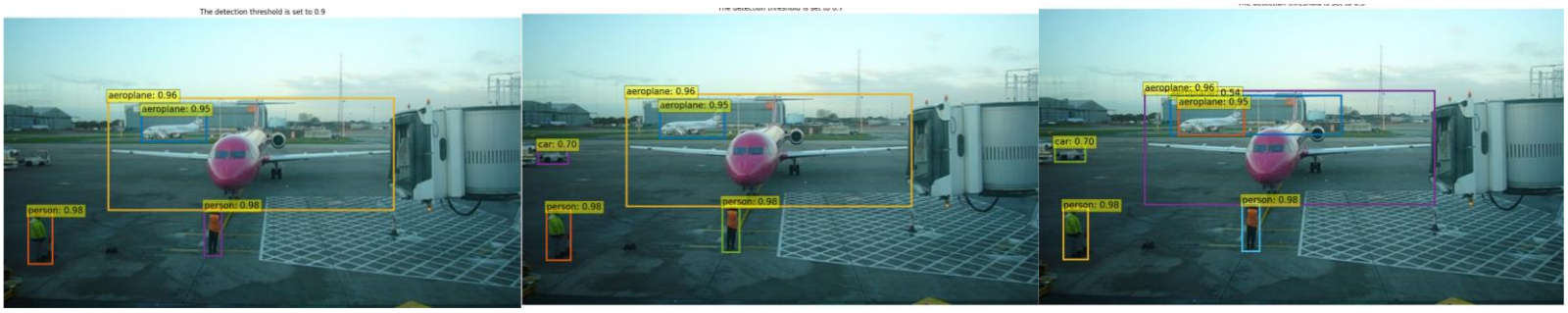


DINO DETR

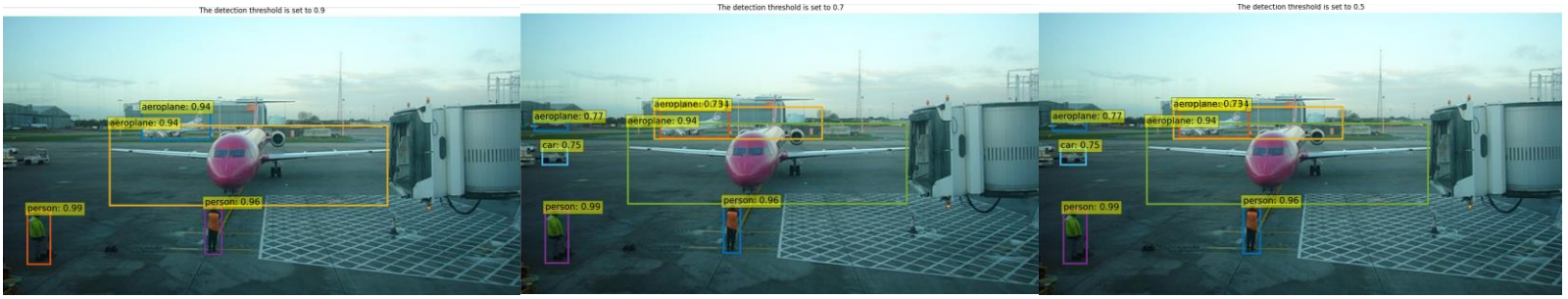
Comparison results
Output of eval.py



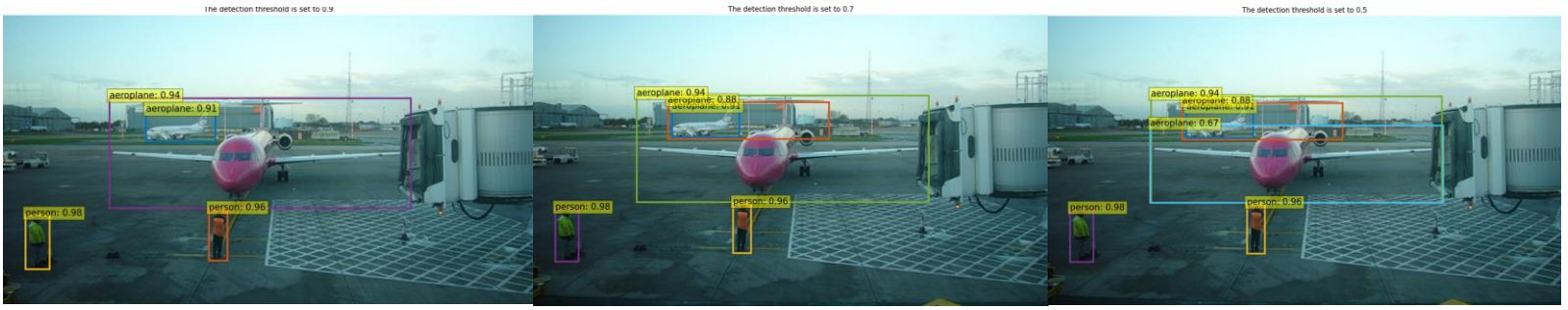
DETR



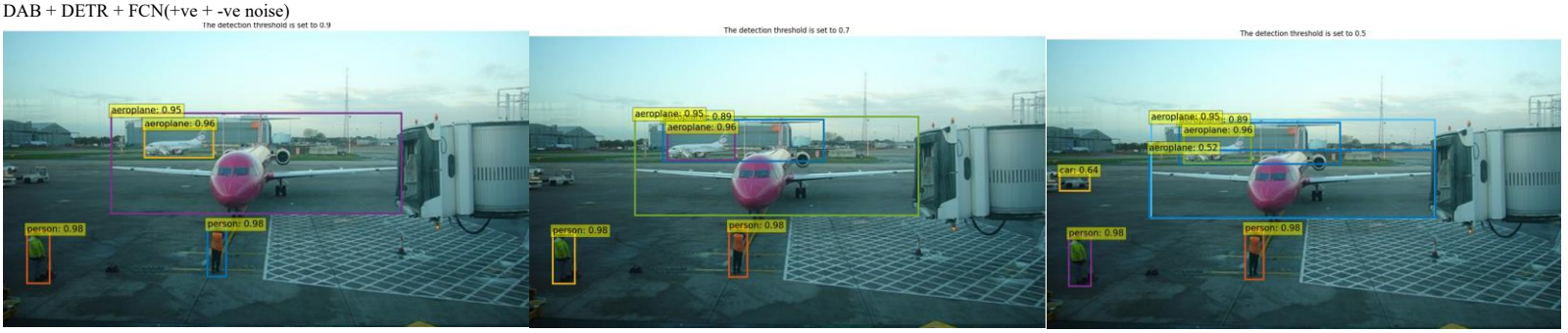
DAB DETR



DAB DETR + FCN



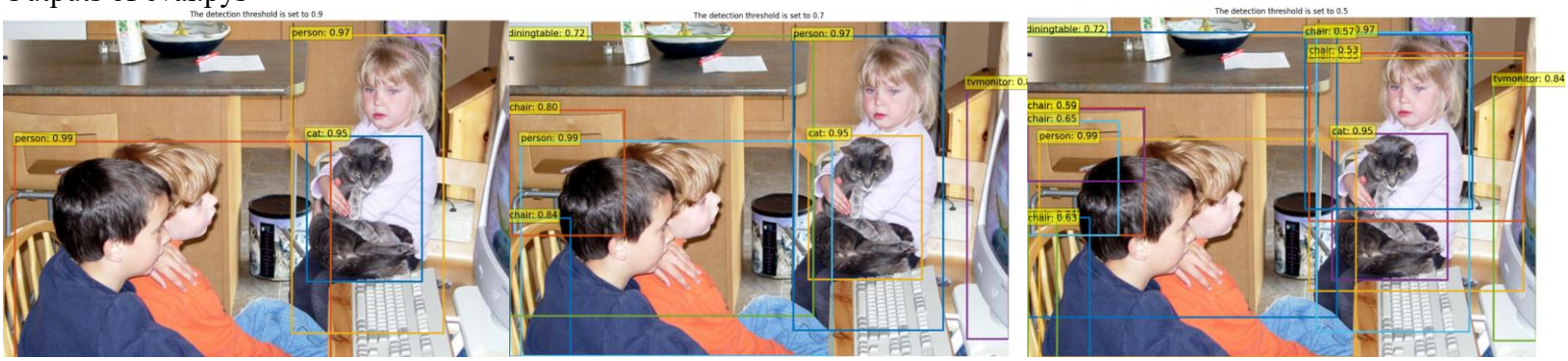
DAB + DETR + FCN(+ve + -ve noise)



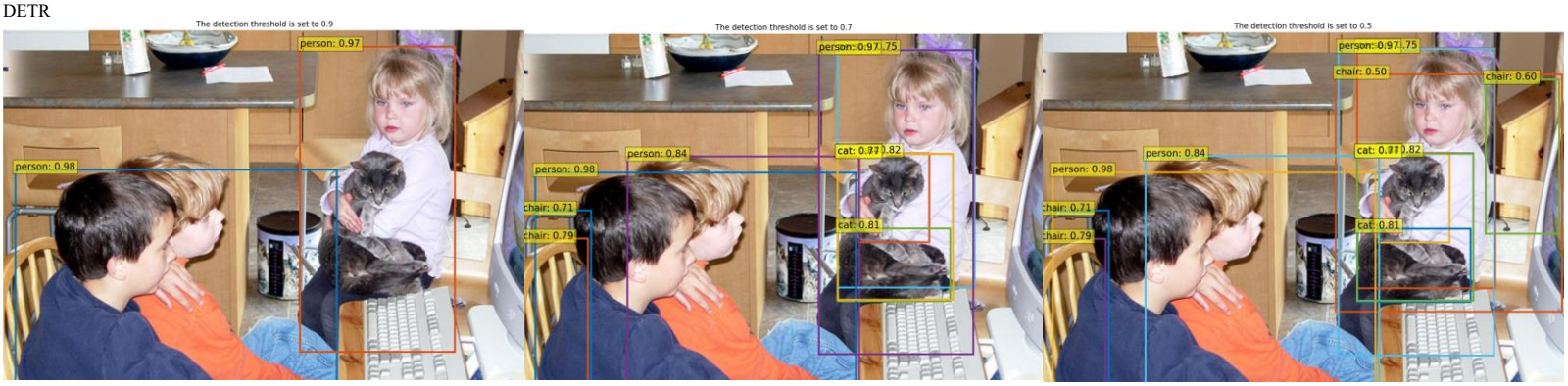
DINO DETR

Comparison of Results

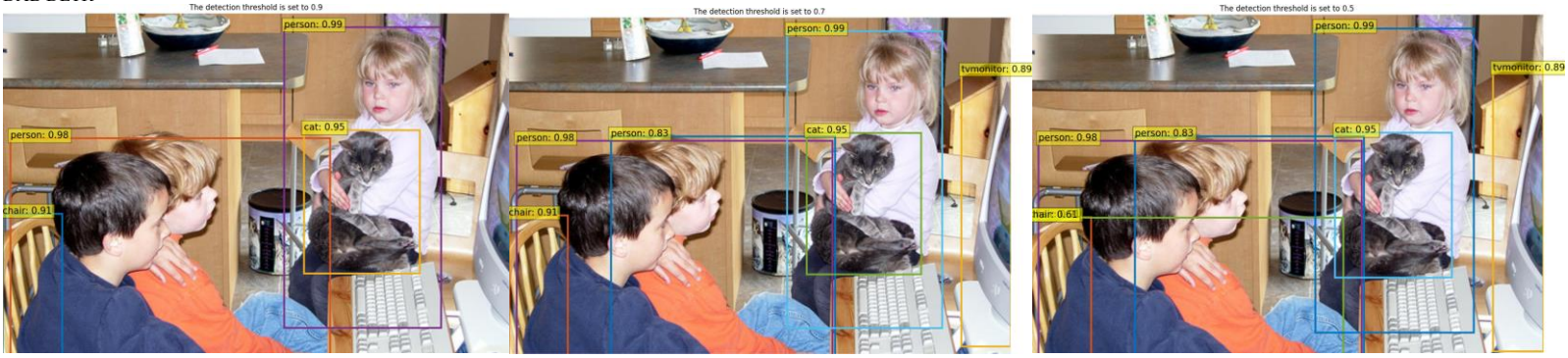
Outputs of eval.pys



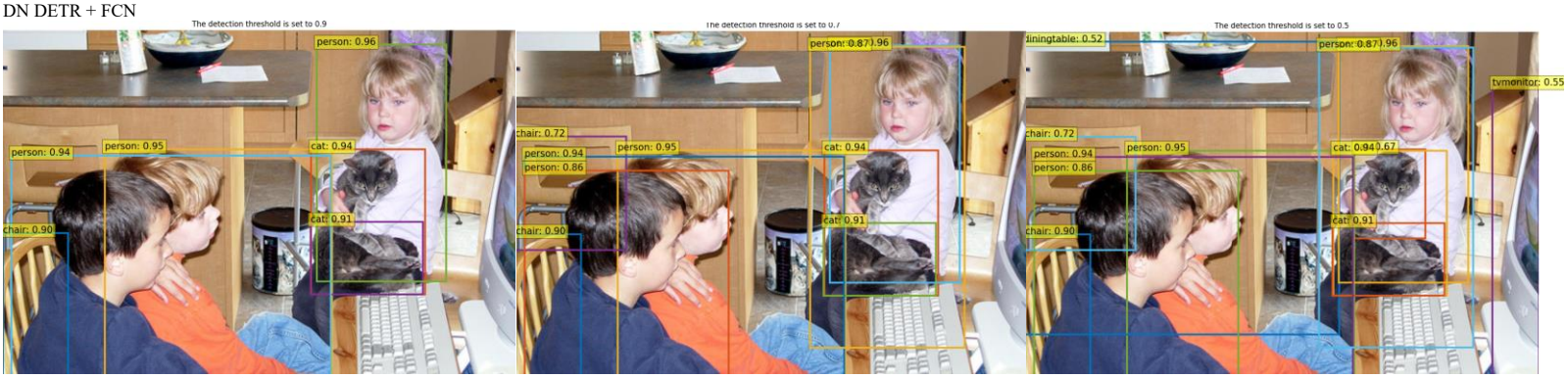
DETR



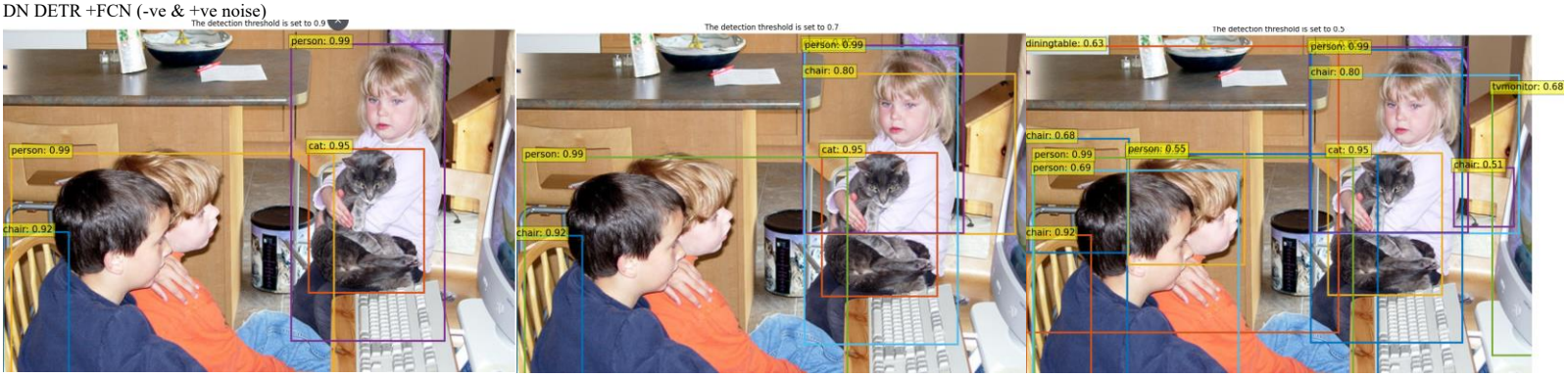
DAB DETR



DN DETR + FCN

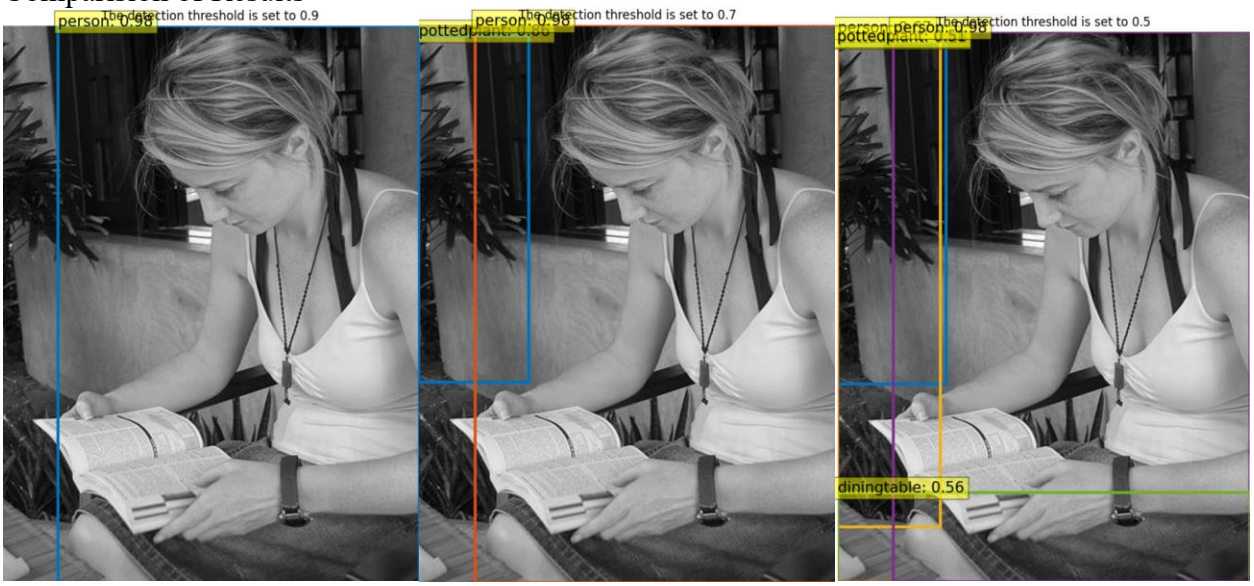


DN DETR +FCN (-ve & +ve noise)

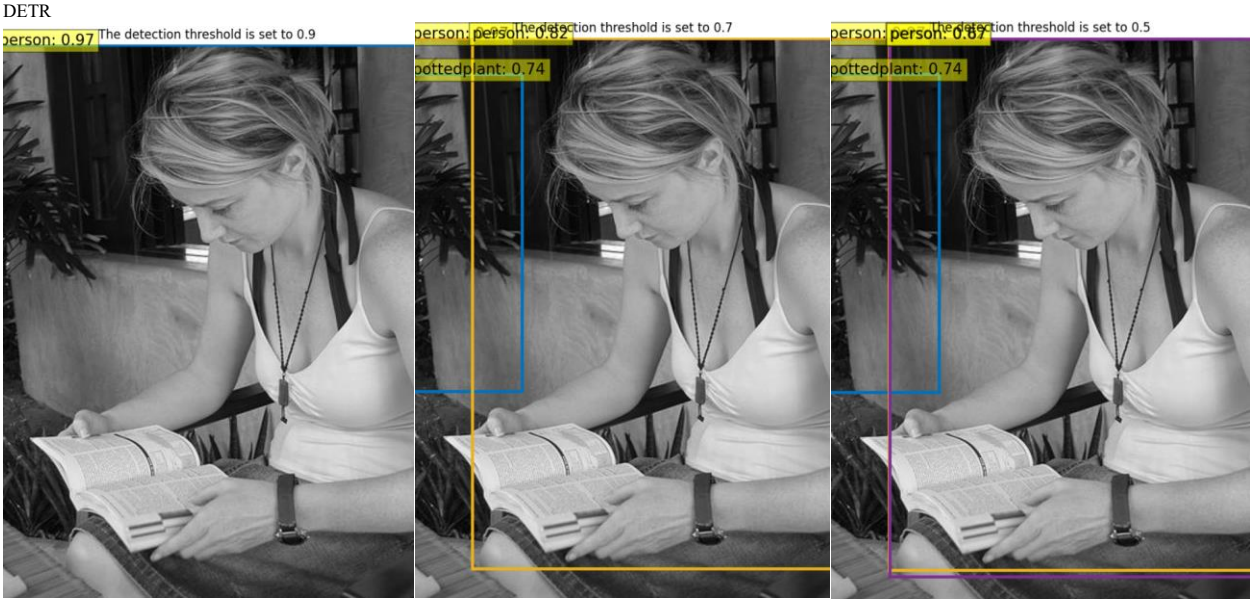


DINO DETR

Comparision of Results

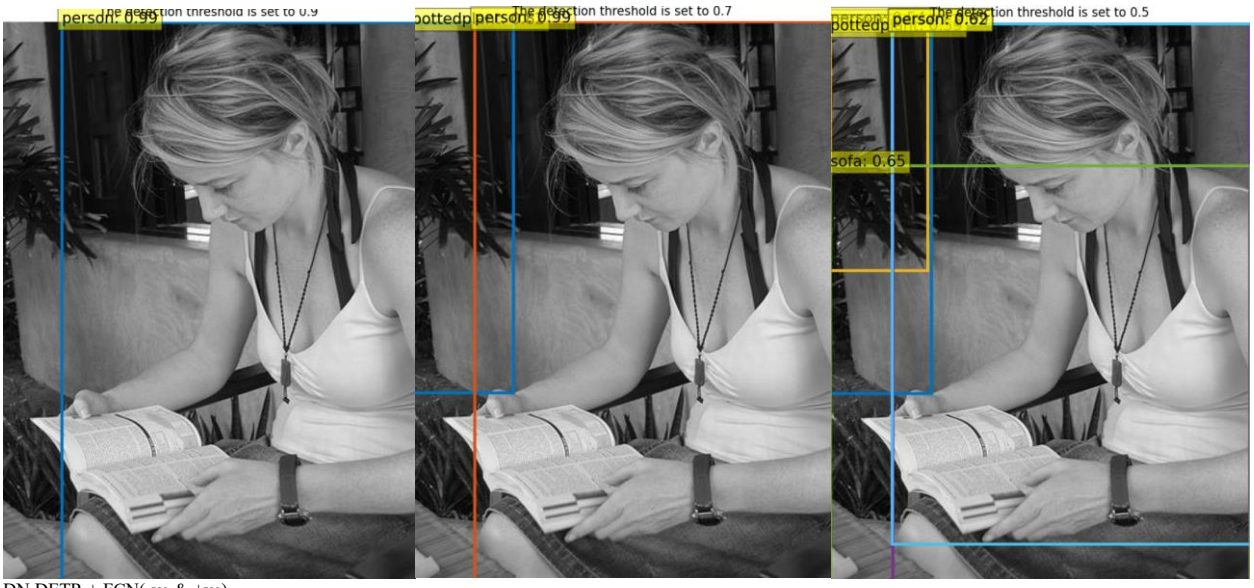


DETR



DAB DETR

DN DETR + FCN



DN DETR + FCN(-ve & +ve)

DINO DETR